

```
=====
=====
CHALLENGE TÉCNICO – DESARROLLADOR FULL STACK SEMI SENIOR
Cliente   : Oxiquim
Nivel     : Semi Senior
Stack     : C# · VB.NET · .NET Framework/.NET Core · Web API · WCF ·
           SQL Server · Oracle · PL/SQL · webMethods · REST/SOAP ·
           HTML5 · CSS3 · JavaScript · jQuery · Bootstrap ·
           Git · Azure DevOps / TFS · Visual Studio 2019-2022
Duración  : 60 minutos
Modalidad: Mixto (Teórico + Diseño + Código)
Fecha     : 2026-05-11
Generado para: Proceso de selección – Vicente Mallea / Francisca
Álvarez
Reclutador: Romina Toffoletti
=====
=====
```

INSTRUCCIÓN PARA EL ENTREVISTADOR – DISEÑO ANTI-IA

Este challenge está diseñado para que NO sea posible responder correctamente usando un asistente de IA en tiempo real. Estrategias aplicadas:

1. PREGUNTAS DE EXPERIENCIA REAL
Cada pregunta técnica exige que el candidato cite un proyecto concreto (empresa, contexto, problema real). La IA genera respuestas genéricas; el entrevistador presiona con follow-ups que verifican autenticidad.
2. FOLLOW-UP SORPRESA VERBAL
Cada pregunta incluye un  FOLLOW-UP que el entrevistador hace VERBALMENTE mientras el candidato responde. El candidato no los conoce de antemano y no puede prepararlos con IA.
3. DISEÑO EN VIVO CON INTERRUPCIONES
Las preguntas de arquitectura y diseño requieren que el candidato piense en tiempo real mientras el entrevistador interrumpe con restricciones nuevas a medida que el candidato habla.
4. CODE CHALLENGE SIN HERRAMIENTAS
El candidato lee y analiza código en papel o pantalla sin acceso a buscadores, IDE ni asistentes de IA. Debe explicar cada bug con su razonamiento técnico.
5. TECNOLOGÍAS ESPECIALIZADAS Y CONTEXTO OXIQUM
webMethods, WCF, TFS, PL/SQL con AUTONOMOUS_TRANSACTION son lo suficientemente específicas para que una respuesta genérica de IA sea fácilmente identificable. El contexto de empresa química (inventario, órdenes de compra, integración SAP) ancla las preguntas a un dominio que no puede responderse solo con teoría.

RE-1 Backend .NET Preguntas A-1, A-2, A-3 + Code Challenge
RE-2 IDE & Herramientas Pregunta WU-2 (Git) + F-1 (Azure DevOps)
RE-3 Base de Datos Preguntas B-1, B-2, B-3 + Code Challenge
RE-4 Arquitectura Preguntas C-1, C-2
RE-5 Integración Preguntas D-1, D-2
RE-6 Frontend Preguntas E-1, E-2

SECCIÓN 0 – WARM-UP (5 min) · No evaluado con puntaje

WU-1: "Cuéntame del proyecto .NET más complejo en el que hayas trabajado. ¿Cuál fue el problema técnico que más trabajo te costó resolver y qué aprendiste de él?"

Señal positiva : nombra empresa, tecnología, bug o decisión concreta.

Señal negativa : respuesta genérica sin fricción ni error propio.

WU-2: "¿Cómo organizas el trabajo en tu repositorio Git cuando eres parte de un equipo de 3-4 desarrolladores? Cuéntame de un merge conflict difícil que hayas tenido que resolver." [RE-2 Git]

 Follow-up: "¿Prefieres git merge o git rebase para integrar una feature branch al main? ¿Por qué y cuándo cambiarías esa preferencia?"

SECCIÓN A – BACKEND .NET (15 min) [RE-1]

Pregunta A-1: C# avanzado – async/await y diferencias .NET Core vs Framework

ENUNCIADO:

Escribeme de memoria la firma de un método async en C# que recibe un userId, consulta la base de datos y retorna una lista de órdenes de

compra. Luego explicame: ¿cuáles son las 3 diferencias más importantes entre desarrollar ese mismo método en .NET Framework 4.8 vs .NET 8?

[El entrevistador puede pedir que lo escriba en papel o en la pizarra.]

● RESPUESTA MÍNIMA ESPERADA (must-mention):

- Firma correcta: `public async Task<List<Order>> GetOrdersAsync(int userId)`
- `await` en operaciones I/O (nunca `.Result/.Wait()` que causa deadlock)
- .NET 8: Minimal APIs, cross-platform, mejor performance de runtime
- .NET Framework 4.8 no corre en Linux/contenedores; .NET 8 sí

📖 EXPLICACIÓN DE REFERENCIA:

Las diferencias clave son: (1) .NET 8 incluye Minimal APIs y permite escribir controllers más livianos sin el overhead de MVC completo; (2) el runtime de .NET 8 tiene mejoras de GC y SIMD que reducen latencia; (3) Dependency Injection está unificado en .NET 8, mientras que en .NET Framework el contenedor DI nativo era limitado y se dependía de Unity/Autofac/Ninject. El peligro más común con `async/await` en .NET Framework era el deadlock por `SynchronizationContext` al mezclar `.Result` con `async`, problema que .NET 8 mitiga con `ConfigureAwait(false)` sistemático o con contextos sin `SynchronizationContext`.

★ RESPUESTA DESTACADA:

Menciona `ConfigureAwait(false)` y por qué importa en bibliotecas vs aplicaciones. Habla del `ValueTask<T>` para hot paths. Menciona `IAsyncEnumerable` para streaming de resultados. Conoce el impacto de capturar el `SynchronizationContext` en contextos ASP.NET Classic.

● RED FLAGS:

- Mezcla `async void` con `async Task` sin explicar cuándo está justificado.
- No sabe qué es un deadlock de `async/await` con `.Result`.
- Cree que .NET Framework y .NET Core son lo mismo.

🎯 FOLLOW-UP SORPRESA:

"En un proyecto real, ¿alguna vez tuviste un deadlock causado por `async/await`? ¿Qué hacía el código y cómo lo diagnosticaste?"

PUNTUACIÓN:

+-----+-----	
-----+	
0 - No apto	No sabe escribir un método async correcto
1 - Básico	Escribe la firma pero no explica diferencias Fw/Core
2 - Competente	Firma correcta + 3 diferencias + menciona riesgo deadlock
3 - Destacado	Agrega <code>ConfigureAwait</code> , <code>ValueTask</code> , <code>IAsyncEnumerable</code>
+-----+-----	
-----+	

Pregunta A-2: Web API + WCF – convivencia y migración

ENUNCIADO:

Oxiquim tiene servicios WCF legacy con contratos existentes. El nuevo equipo necesita consumirlos desde una app React. ¿Cómo los expondrías como endpoints REST sin reescribir los servicios WCF desde cero?

Diseña la solución en voz alta mientras el entrevistador hace preguntas.

[El entrevistador interrumpe durante la respuesta con las preguntas de follow-up marcadas abajo.]

● RESPUESTA MÍNIMA ESPERADA (must-mention):

- Capa adaptadora: un Web API controller que internamente instancia y llama al canal WCF usando `ChannelFactory<TService>`
- El contrato WCF (`IService`) no se modifica; el controller traduce DTOs
- El endpoint WCF queda en red interna; solo el Web API es público

📖 EXPLICACIÓN DE REFERENCIA:

La estrategia Facade/Adapter crea un Web API (.NET 6/8) que actúa como proxy hacia los servicios WCF internos usando `ChannelFactory<T>` o el cliente generado con `svcutil/dotnet-svcutil`. El Web API define sus propios DTOs de request/response y mapea desde/hacia los tipos WCF internos. Esto desacopla el contrato externo (REST) del contrato interno

(WCF SOAP), permitiendo migrar los WCF gradualmente sin impactar a los consumidores REST. Para seguridad, el endpoint WCF debe estar en la red interna; solo el Web API tiene exposición externa con JWT o API Key.

★ RESPUESTA DESTACADA:

Propone CoreWCF como alternativa moderna para migrar los servicios WCF a .NET 6/8 manteniendo el contrato SOAP. Menciona el patrón Strangler Fig para migración gradual. Habla de la gestión de sesiones y transacciones distribuidas entre el Web API y el WCF.

● RED FLAGS:

- Propone reescribir todos los WCF de inmediato sin evaluar el costo.
- No considera la seguridad del endpoint WCF interno.
- No conoce `ChannelFactory` ni cómo consumir WCF programáticamente.

🌀 FOLLOW-UP SORPRESA (interrumpir durante la respuesta):

"¿Qué pasa con la seguridad? El WCF usa Windows Authentication (`wsHttpBinding`). El Web API externo tiene JWT. ¿Cómo pasas la identidad del usuario del JWT al servicio WCF sin que todo corra como la cuenta de servicio del servidor?"

PUNTUACIÓN:

+-----+-----	
-----+	
0 - No apto	No sabe qué es WCF o no puede diseñar la integración
1 - Básico	Propone reescribir o solo menciona el adapter sin detalle
2 - Competente	Facade + ChannelFactory + seguridad del endpoint interno
3 - Destacado	+ CoreWCF + Strangler Fig + manejo de identidad JWT
-> WCF	
+-----+-----	
-----+	

Pregunta A-3: VB.NET legacy – mantenimiento y migración

ENUNCIADO:

Encuentras este bloque en un módulo VB.NET legacy de Oxiquim:

```

On Error GoTo ErrorHandler
Dim conn As New SqlConnection(connStr)
conn.Open()
' ... lógica de negocio ...
Exit Sub
ErrorHandler:
MsgBox "Error: " & Err.Description

```

¿Qué problemas tiene este patrón? Si el cliente pide agregar una nueva validación a este módulo hoy, ¿cómo lo haces sin romper nada y sin reescribir todo?

● RESPUESTA MÍNIMA ESPERADA (must-mention):

- On Error GoTo no libera recursos: conn nunca se cierra si hay excepción
- MsgBox en lógica de negocio mezcla UI con lógica (viola separación)
- Corrección mínima: Try/Catch/Finally con conn en el Finally
- Migración gradual: primero el manejo de errores, luego extracción

📖 EXPLICACIÓN DE REFERENCIA:

On Error GoTo es el manejo de errores de VB6 arrastrado a VB.NET. En VB.NET moderno se usa Try/Catch/Finally igual que C#. El problema de recursos no liberados se resuelve con Using (equivalente al using de C#). Para una migración gradual sin romper funcionalidad: (1) envolver en Try/Catch/Finally sin cambiar la lógica; (2) extraer la lógica de negocio a un método que retorne un resultado tipado; (3) reemplazar MsgBox por logging estructurado; (4) migrar a C# si justifica el esfuerzo.

★ RESPUESTA DESTACADA:

Menciona que Using en VB.NET garantiza la disposición del recurso.

Propone crear tests de regresión antes de tocar el módulo legacy.
Habla de la interoperabilidad VB.NET/C# en la misma solución .NET
(ambos compilan a IL y pueden llamarse mutuamente).

● RED FLAGS:

- Propone reescribir todo el módulo sin justificación de costo/beneficio.
- No detecta el problema de la conexión no cerrada.
- No conoce Try/Catch/Finally en VB.NET.

🔗 FOLLOW-UP SORPRESA:

"¿En qué situación coexistiría un proyecto VB.NET con uno C# en la misma solución .NET? ¿Cómo compartirías una librería de utilidades entre ambos sin duplicar código?"

PUNTUACIÓN:

+-----+-----	
-----+	
0 - No apto	No detecta el problema de recursos o desconoce VB.NET
1 - Básico	Identifica el problema pero solo propone reescribir
2 - Competente	Try/Catch/Finally + Using + migración gradual
3 - Destacado	+ interop VB.NET/C# + tests de regresión previos
+-----+-----	
-----+	

SECCIÓN B – BASE DE DATOS (12 min) [RE-3]

Pregunta B-1: SQL Server – optimización de queries en producción

ENUNCIADO:

En producción tienes esta query que demora 45 segundos en una tabla de 10 millones de filas en SQL Server:

```
SELECT o.Id, o.Total, c.Nombre
FROM Ordenes o
INNER JOIN Clientes c ON o.ClienteId = c.Id
WHERE o.Estado = 'Activo'
AND YEAR(o.FechaCreacion) = 2024
```

El execution plan muestra un Clustered Index Scan sobre Ordenes.

Describe tus primeros 3 pasos para diagnosticar y resolver el problema.

● RESPUESTA MÍNIMA ESPERADA (must-mention):

- YEAR(o.FechaCreacion) impide usar un índice (función sobre columna, non-sargable); reescribir como rango de fechas explícito
- Crear índice no agrupado compuesto en (Estado, FechaCreacion) con INCLUDE de las columnas necesarias para evitar Key Lookup
- Revisar estadísticas: UPDATE STATISTICS o REBUILD INDEX si están desactualizadas

📖 EXPLICACIÓN DE REFERENCIA:

El problema raíz es que YEAR() es non-sargable: el optimizador no puede usar un índice en FechaCreacion porque debe aplicar la función a cada fila. Reescribir como:

FechaCreacion >= '2024-01-01' AND FechaCreacion < '2025-01-01'

es la corrección más impactante. Luego, el índice compuesto (Estado, FechaCreacion) INCLUDE (Total, ClienteId) cubre la query sin Key Lookup. Si Estado tiene baja selectividad (muchos 'Activo'), invertir

el orden del índice a (FechaCreacion, Estado) puede mejorar la selectividad.

Verificar también el plan estimado vs real para detectar diferencias de cardinalidad que indiquen estadísticas desactualizadas.

★ RESPUESTA DESTACADA:

Menciona filtered indexes para WHERE Estado = 'Activo' (índice parcial solo sobre filas activas). Habla de Query Store para detectar regresiones

de performance a lo largo del tiempo. Diferencia entre Index Seek vs Index Scan y cuándo SQL Server opta por Scan aunque exista el índice.

● RED FLAGS:

- Solo dice "crear un índice en FechaCreacion" sin identificar el YEAR().
- No sabe qué es un execution plan.
- Propone poner índices en todas las columnas del WHERE sin evaluar.

🎯 FOLLOW-UP SORPRESA:

"Si agrego el índice y la query mejora de 45 seg a 2 seg, pero el DBA me dice que los INSERTs/UPDATEs se desaceleraron un 30%, ¿cómo tomo la decisión de mantener o eliminar el índice? ¿Qué información necesito?"

PUNTUACIÓN:

```
+-----+-----+
-----+
| 0 - No apto      | No sabe leer un execution plan o no detecta el
YEAR()            |
| 1 - Básico       | Detecta el problema pero solo propone agregar un
índice            |
| 2 - Competente   | Non-sargable + índice INCLUDE + estadísticas
|
```

| 3 - Destacado | + filtered index + Query Store + trade-off
insert/update |
+-----+-----+
-----+

Pregunta B-2: Stored Procedures + Triggers + Deadlocks

ENUNCIADO:

Tienes un trigger AFTER UPDATE en la tabla Transacciones de Oxiquim que escribe en una tabla de Auditoría. Simultáneamente, un Stored Procedure de cierre mensual también escribe en Transacciones y luego en Auditoría en orden inverso. Los lunes en la noche, cuando corre el cierre, el sistema genera deadlocks. ¿Cómo lo diagnosticas y resuelves sin eliminar el trigger?



RESPUESTA MÍNIMA ESPERADA (must-mention):

- El deadlock ocurre por orden de bloqueo inverso: trigger adquiere Transacciones -> Auditoría; SP adquiere Auditoría -> Transacciones
- Diagnóstico: Extended Events / SQL Server Profiler con deadlock_graph
- Solución principal: unificar el orden de acceso a tablas (siempre Transacciones -> Auditoría) en el SP y en el trigger



EXPLICACIÓN DE REFERENCIA:

Un deadlock clásico ocurre cuando dos procesos se bloquean mutuamente esperando recursos que el otro tiene. Para diagnosticarlo, el deadlock graph en Extended Events muestra exactamente qué proceso esperaba qué recurso. La solución más robusta es garantizar que TODOS los accesos a Transacciones y Auditoría ocurran siempre en el mismo orden.

Alternativas:

(1) mover la lógica de auditoría del trigger a Service Broker async para

sacarla de la transacción principal; (2) usar READ_COMMITTED_SNAPSHOT isolation level para reducir conflictos de lectura; (3) revisar si el trigger puede reemplazarse con temporal tables (SQL Server 2016+) que implementan auditoría sin trigger.



RESPUESTA DESTACADA:

Menciona Service Broker para auditoría asíncrona. Habla de READ_COMMITTED_SNAPSHOT y sus implicaciones en tempdb (versioning). Propone temporal tables (system-versioned) como alternativa moderna. Conoce el impacto de NOLOCK y cuándo NO usarlo para resolver deadlocks.



RED FLAGS:

- Propone eliminar el trigger como única solución.
- No sabe diagnosticar un deadlock (no conoce Extended Events).
- Propone NOLOCK como solución sin entender que introduce dirty reads.



FOLLOW-UP SORPRESA:

"¿Cuándo usarías un trigger y cuándo preferirías manejar la auditoría a nivel de aplicación o de ORM? ¿Qué criterio técnico define esa decisión en un equipo como el de Oxiquim?"

PUNTUACIÓN:

+-----+-----	
-----+	
0 - No apto	No entiende qué es un deadlock o cómo ocurre
1 - Básico	Identifica el problema pero solo propone eliminar trigger
2 - Competente	Deadlock graph + orden de bloqueo + unificación de acceso
3 - Destacado	+ Service Broker async + temporal tables + RCSI
+-----+-----	
-----+	

Pregunta B-3: Oracle / PL/SQL

ENUNCIADO:

¿Qué es una AUTONOMOUS_TRANSACTION en PL/SQL de Oracle y en qué escenario concreto la usarías en Oxiquim? ¿Cuál es el principal riesgo si se usa incorrectamente?



RESPUESTA MÍNIMA ESPERADA (must-mention):

- PRAGMA AUTONOMOUS_TRANSACTION ejecuta un bloque PL/SQL en una transacción independiente del contexto que la invocó
- Caso de uso clásico: logging o auditoría que debe persistir incluso si la transacción principal hace ROLLBACK
- Riesgo: puede generar inconsistencias porque los cambios persisten aunque la transacción padre falle (datos huérfanos)



EXPLICACIÓN DE REFERENCIA:

PRAGMA AUTONOMOUS_TRANSACTION permite que un procedimiento PL/SQL haga COMMIT o ROLLBACK sin afectar la transacción que lo invocó. El caso de uso más legítimo es un procedimiento de log de errores que registra el error en una tabla aunque la transacción de negocio haga ROLLBACK. El riesgo es que si la factura falla y hace ROLLBACK, pero el procedimiento de auditoría con AUTONOMOUS_TRANSACTION ya hizo COMMIT, quedará un registro de auditoría de una factura que no existe. En SQL Server, el equivalente funcional es Service Broker o xp_logevent para log fuera de transacción.



RESPUESTA DESTACADA:

Distingue entre el modelo de transacciones Oracle (sin autocommit por defecto) vs SQL Server (autocommit por defecto a nivel de statement). Menciona SAVEPOINT en Oracle para rollback parcial. Compara el

● RED FLAGS:

-  FOLLOW-UP SORPRESA:

PUNTUACIÓN:

SECCIÓN C – ARQUITECTURA Y PATRONES DE DISEÑO (8 min) [RE-4]

ENUNCIADO:

[El entrevistador interrumpe durante la respuesta.]

- RESPUESTA MÍNIMA ESPERADA (must-mention):
 - Presentación: Controller o ViewModel que recibe DTOs, nunca entidades de dominio directamente
 - Negocio: Service/Manager con la lógica de validación y reglas (ej: límite de crédito del proveedor, aprobación de montos)
 - Datos: Repository que encapsula el acceso a SQL Server; la capa de negocio no conoce SqlConnection ni queries

- Dependencias apuntan hacia adentro: Presentación -> Negocio -> Datos

📖 EXPLICACIÓN DE REFERENCIA:

El patrón multicapas separa responsabilidades para que cambiar la BD no impacte la capa de negocio, y cambiar la UI no impacte la lógica.

Los contratos entre capas son interfaces: `IOrderRepository`, `IOrderService`.

El Controller recibe y retorna DTOs (`CreateOrderRequest`, `OrderResponse`),

nunca entidades ORM. La capa de datos implementa el repositorio; si usa Entity Framework, el `DbContext` vive en la capa de datos y es inyectado por DI. Las interfaces permiten testear la capa de negocio con mocks sin necesitar la base de datos.

★ RESPUESTA DESTACADA:

Propone una capa de Application Services separada de Domain Services. Menciona Unit of Work para coordinar múltiples repositorios en una transacción. Habla de la diferencia entre anemic model y rich domain model y cuándo cada uno aplica.

● RED FLAGS:

- Pone lógica de negocio en el Controller.
- Pone queries SQL directamente en la capa de negocio.
- No puede explicar por qué las interfaces son importantes.

🎯 FOLLOW-UP SORPRESA (interrumpir mientras describe):

"¿La validación del RUT del proveedor va en la capa de presentación, de negocio o de datos? ¿Por qué? Si mañana necesitas la misma validación en un proceso batch sin UI, ¿dónde queda ese código?"

PUNTUACIÓN:

+-----+-----	
-----+	
0 - No apto	No puede separar responsabilidades entre capas
1 - Básico	Identifica las capas pero mezcla responsabilidades
2 - Competente	Capas claras + interfaces + DTOs separados de entidades
3 - Destacado	+ Unit of Work + Application vs Domain Services
+-----+-----	
-----+	

Pregunta C-2: Patrones de diseño aplicados

ENUNCIADO:

El sistema de OxiQuim genera reportes en 5 formatos: PDF, Excel, CSV, HTML y JSON. Cada vez que llega un nuevo formato, el desarrollador

modifica el mismo método gigante con un switch/case que ya tiene 400 líneas. ¿Qué patrón de diseño aplicas y cómo queda la estructura en C#?

● RESPUESTA MÍNIMA ESPERADA (must-mention):

- Patrón Strategy: una interfaz IReportGenerator con método Generate()
- Una implementación por formato: PdfReportGenerator, ExcelReportGenerator
- Factory o DI para resolver el generador según el formato solicitado
- El código cliente no cambia cuando se agrega un nuevo formato (principio Open/Closed)

📖 EXPLICACIÓN DE REFERENCIA:

Strategy elimina el switch/case reemplazándolo con polimorfismo. La interfaz IReportGenerator<T> tiene un método Generate(T data): byte[]. Cada implementación encapsula la lógica del formato específico. El Factory (o un Dictionary<string, IReportGenerator> en DI) resuelve qué implementación usar según el tipo. Agregar un nuevo formato requiere solo

crear una nueva clase que implemente la interfaz y registrarla en el contenedor DI, sin modificar código existente. Si los reportes comparten pasos comunes (obtener datos, transformar, serializar), el patrón Template Method complementa bien al Strategy.

★ RESPUESTA DESTACADA:

Combina Strategy + Template Method para pasos compartidos. Propone Abstract Factory si los reportes tienen variantes por módulo (Compras PDF, Inventario PDF). Habla de cómo registrar las estrategias en el contenedor DI de .NET sin switch/case en el Factory.

● RED FLAGS:

- Propone Strategy pero sigue usando switch/case en el Factory.
- No puede escribir la interfaz en C# de memoria.
- Confunde Strategy con State.

🎯 FOLLOW-UP SORPRESA:

"Si en 6 meses hay 20 formatos y el equipo registra cada uno en el contenedor DI manualmente, ¿cómo evitas que el archivo de configuración de DI crezca sin control? ¿Cómo automatizarías el registro de nuevas estrategias?"

PUNTUACIÓN:

+-----+-----	
-----+	
0 - No apto	No conoce patrones de diseño o no puede aplicar Strategy
1 - Básico	Nombra Strategy pero no puede describir la estructura
2 - Competente	Interfaz + implementaciones + Factory/DI + Open/Closed
3 - Destacado	+ Template Method + registro automático via Reflection/DI

+-----+-----+
-----+

SECCIÓN D – INTEGRACIÓN Y SERVICIOS (8 min) [RE-5]

Pregunta D-1: webMethods y orquestación de servicios

ENUNCIADO:

Oxiquim usa webMethods como middleware de integración entre el sistema de inventario (.NET) y el sistema de compras SAP, mediante mensajes XML.

Diseña el flujo cuando el sistema .NET detecta stock bajo y debe generar

una orden de reposición en SAP. ¿Qué componentes de webMethods participan

y cómo garantizas que la orden no se duplique si hay un fallo en el envío?

● RESPUESTA MÍNIMA ESPERADA (must-mention):

- El sistema .NET publica el evento/mensaje en webMethods (via HTTP o JMS)
- webMethods orquesta: valida, transforma el mensaje y lo entrega al adapter de SAP (SAP Adapter con BAPI/IDOC)
- Idempotencia: correlationId único por mensaje; webMethods verifica si ese ID ya fue procesado antes de enviar a SAP (deduplicación)

📖 EXPLICACIÓN DE REFERENCIA:

webMethods Integration Server funciona como ESB: recibe eventos via HTTP/AS2/JMS, aplica transformaciones (Integration Objects, XSLT), y los entrega a sistemas destino via adaptadores nativos (SAP Adapter, JDBC

Adapter). Para evitar duplicados en caso de fallo: (1) el mensaje .NET incluye un UUID correlationId; (2) webMethods verifica en su Document Store si ese correlationId ya fue procesado antes de enviarlo a SAP; (3) si SAP devuelve error, webMethods puede reintentar con backoff sin duplicar si el BAPI de SAP es idempotente. La garantía at-least-once vs exactly-once define la estrategia de idempotencia requerida.

★ RESPUESTA DESTACADA:

Habla de los paradigmas request-reply vs publish-subscribe en webMethods.

Menciona el Document Model de webMethods para mapeo de campos entre estructuras .NET y IDOC de SAP. Propone monitoreo con webMethods Monitor

para detectar mensajes en estado de error en las colas.

● RED FLAGS:

- No tiene concepto de ESB ni de orquestación de servicios.
- No considera el caso de fallo y duplicados.
- Confunde webMethods con un simple broker de mensajes (RabbitMQ/Kafka).

🔗 FOLLOW-UP SORPRESA:

"Si SAP rechaza la orden porque el proveedor no existe en su catálogo, ¿cómo diseñas el flujo de error para que el usuario del sistema .NET sea notificado y pueda corregir el dato? ¿Quién es responsable de ese flujo de compensación en el equipo?"

PUNTUACIÓN:

+-----+-----	
-----+	
0 - No apto	No conoce webMethods ni conceptos de
ESB/orquestación	
1 - Básico	Conoce el concepto pero no puede diseñar el flujo
2 - Competente	Flujo completo + correlationId + deduplicación +
retry	
3 - Destacado	+ Document Model SAP + Monitor + flujo de
compensación	
+-----+-----	
-----+	

Pregunta D-2: REST vs SOAP – criterio de selección

ENUNCIADO:

¿Cuándo en 2025 aún elegirías SOAP sobre REST para una nueva integración en Oxiquim? Dame un ejemplo concreto. Y cuéntame del último servicio SOAP que consumiste: ¿qué problema encontraste con el WSDL o el cliente generado en .NET?

● RESPUESTA MÍNIMA ESPERADA (must-mention):

- SOAP es preferible cuando el contrato formal (WSDL) es crítico, cuando
 - se necesita WS-Security para firma de mensajes, o cuando el sistema destino solo soporta SOAP (SAP, bancos, SII)
- Problema clásico con WSDL: tipos complejos que no mapean bien a C#, o WSDL con múltiples endpoints que el generador combina incorrectamente

📖 EXPLICACIÓN DE REFERENCIA:

SOAP sigue justificado en integraciones enterprise donde el contrato es

legal o regulatorio (SII en Chile, PREVIRED, bancos), cuando se requiere
WS-AtomicTransaction para transacciones distribuidas, o cuando el sistema
destino es legacy y solo expone WSDL. Los problemas más comunes al generar el cliente con svcutil o dotnet-svcutil son: namespaces conflictivos en WSDL con múltiples schemas, tipos con nombres duplicados,
y endpoints con política de seguridad que el generador no resuelve automáticamente. La solución es editar el reference.cs generado o usar un cliente HTTP manual con XmlSerializer para los casos más complejos.

★ RESPUESTA DESTACADA:

Menciona OpenAPI/Swagger como el equivalente moderno al WSDL para REST. Habla de WS-Security con certificados X.509 para firma de mensajes. Menciona GraphQL como alternativa cuando el consumidor necesita control sobre los campos retornados.

● RED FLAGS:

- "SOAP nunca se usa hoy" – desconoce el contexto enterprise chileno.
- Nunca trabajó con un WSDL real.
- No puede explicar la diferencia entre SOAP envelope y HTTP body REST.

🔗 FOLLOW-UP SORPRESA:

"¿Cómo depurarías un servicio SOAP en producción cuando el cliente .NET lanza un SoapException genérico sin detalle? ¿Qué herramientas usas para ver el XML del request/response real?"

PUNTUACIÓN:

+-----+-----	
-----+	
0 - No apto	No puede dar un caso de uso válido para SOAP en 2025
1 - Básico	Da el caso de uso pero nunca trabajó con WSDL real
2 - Competente	Caso de uso + problemas reales con WSDL + cliente generado
3 - Destacado	+ WS-Security + debugging de envelope + OpenAPI vs WSDL
+-----+-----	
-----+	

SECCIÓN E – FRONTEND (6 min) [RE-6]

Pregunta E-1: JavaScript/jQuery – performance con grandes volúmenes

ENUNCIADO:

Una vista en Razor/HTML con jQuery carga 50.000 filas de la tabla de inventario y el navegador se congela al renderizarlas. El usuario pide poder filtrar por descripción mientras escribe. ¿Cómo lo resuelves?

● RESPUESTA MÍNIMA ESPERADA (must-mention):

- Paginación server-side: el backend devuelve solo la página actual (20-50 filas) y el total; el cliente navega entre páginas
- Filtro con debounce en el input: no llamar al servidor en cada keystroke (setTimeout con 300-400ms)
- Alternativa si se requiere scroll continuo: virtual scrolling

📖 EXPLICACIÓN DE REFERENCIA:

El problema es DOM thrashing: el navegador no puede renderizar 50.000 nodos <tr> eficientemente. La solución es server-side pagination con un endpoint GET /api/inventario?page=1&pageSize=50&filter=texto. El debounce evita 50 llamadas al servidor mientras el usuario escribe: `var timer; $('#filter').keyup(function() { clearTimeout(timer); timer = setTimeout(loadData, 300); });` Si el cliente insiste en client-side, se usa clusterize.js o una tabla virtual que solo renderiza las filas visibles en el viewport.

★ RESPUESTA DESTACADA:

Menciona DataTables.js con server-side processing como solución pragmática. Propone Web Workers para filtrado client-side sin bloquear el UI thread. Habla de la diferencia de performance entre innerHTML batch insert vs appendChild en loop.

● RED FLAGS:

- Propone cargar todos los datos pero ocultarlos con display:none.
- No conoce debounce.
- Propone "usar React" sin entender que el problema es el volumen de datos.

🎯 FOLLOW-UP SORPRESA:

"El cliente pide que la URL cambie cuando el usuario aplica un filtro para poder compartir el link. ¿Cómo lo implementas con jQuery sin recargar la página?"

PUNTUACIÓN:

+-----+-----	
-----+	
0 - No apto	No tiene solución para el performance del DOM
1 - Básico	Propone paginación pero no implementa el debounce
2 - Competente	Server-side pagination + debounce + virtual scrolling

| 3 - Destacado | + DataTables server-side + Web Workers + History API
URL |
+-----+-----+
-----+

Pregunta E-2: CSS3 / Bootstrap – diseño responsivo

ENUNCIADO:

Un junior hizo este HTML con Bootstrap 4 y en desktop se ve bien, pero en móvil las columnas se superponen:

```
<div class="row">
  <div class="col-2">RUT</div>
  <div class="col-2">Nombre</div>
  <div class="col-2">Email</div>
  <div class="col-2">Teléfono</div>
  <div class="col-2">Acciones</div>
</div>
```

¿Qué tiene mal y cómo lo corregirías? ¿Cuándo usarías CSS Grid o Flexbox en lugar del sistema de grilla de Bootstrap?

● RESPUESTA MÍNIMA ESPERADA (must-mention):

- col-2 es fijo en todos los breakpoints; para móvil debe apilarse: col-12 col-sm-6 col-md-4 col-lg-2
- Bootstrap usa un grid de 12 columnas; $5 \times \text{col-2} = 10$, correcto en anchura, pero sin breakpoints responsivos las columnas no colapsan
- CSS Grid/Flexbox cuando se necesita control fino que las clases de Bootstrap no cubren (layouts asimétricos, gaps específicos)

📖 EXPLICACIÓN DE REFERENCIA:

El error es usar solo col-2 sin especificar breakpoints. La corrección: col-12 (apilado en mobile) col-sm-6 (2 col en tablet pequeña) col-md-4 (3 col en tablet) col-lg-2 (5 col en desktop). Para listas tabulares en móvil, la mejor UX es colapsar columnas secundarias con d-none d-md-block

y mostrar solo las esenciales. CSS Grid es preferible a Bootstrap cuando

el diseño tiene filas con alturas dinámicas o el diseñador especifica un

layout que el grid de 12 columnas no puede representar.

★ RESPUESTA DESTACADA:

Menciona Bootstrap 5 (elimina jQuery como dependencia) vs Bootstrap 4. Habla de container-queries (CSS moderno) para layouts basados en el tamaño del contenedor. Propone d-none d-md-block para ocultar columnas secundarias en móvil.

● RED FLAGS:

- No sabe para qué sirven los breakpoints de Bootstrap.
- Propone media queries manuales sin conocer el sistema de grilla.
- No puede explicar qué es el grid de 12 columnas.

 FOLLOW-UP SORPRESA:

"El cliente pide que en móvil la tabla cambie a tarjetas apiladas en lugar de columnas. ¿Cómo lo harías con Bootstrap sin reescribir el HTML?"

PUNTUACIÓN:

+-----+-----	
-----+	
0 - No apto	No entiende el sistema de grilla de Bootstrap
1 - Básico	Agrega breakpoints pero no explica la lógica
2 - Competente	col-12/sm/md/lg + caso de uso Bootstrap vs CSS Grid
3 - Destacado	+ container-queries + d-none responsive + Bootstrap
5	
+-----+-----	
-----+	

SECCIÓN F – HERRAMIENTAS (3 min) [RE-2]

Pregunta F-1: Azure DevOps / TFS – pipeline CI/CD

ENUNCIADO:

En tu proyecto anterior o actual, ¿cómo era el pipeline de CI/CD en Azure DevOps o TFS? ¿Qué validaciones existían antes de que un cambio llegara a producción? Dame un ejemplo de un problema que detectaste gracias al pipeline antes de llegar a producción.

 RESPUESTA MÍNIMA ESPERADA (must-mention):

- Build automatizado en cada PR o push a la rama principal
- Al menos: compilación + tests unitarios como gate de calidad
- Deploy a ambiente de QA antes de producción (con aprobación manual o automática según la configuración)

 RED FLAGS:

- Deploy manual desde Visual Studio directamente a producción.
- No tiene tests como parte del pipeline.
- No sabe la diferencia entre Azure DevOps (cloud) y TFS (on-premise).

 FOLLOW-UP SORPRESA:

"Si el pipeline falla en el paso de tests a las 10 PM y el cliente necesita el hotfix en producción para las 8 AM, ¿qué haces? ¿Tienes

algún proceso de emergencia y qué riesgos asumes al saltarte el pipeline?"

PUNTUACIÓN:

+-----+-----	
-----+	
0 - No apto	Deploy manual, sin pipeline de CI/CD
1 - Básico	Build automático pero sin tests ni gates de calidad
2 - Competente	Build + tests + deploy a QA + aprobación antes de prod
3 - Destacado	+ proceso de hotfix emergencia + rollback plan definido
+-----+-----	
-----+	

SECCIÓN G – CODE CHALLENGE (12 min) [RE-1 + RE-3]



INSTRUCCIÓN AL ENTREVISTADOR:

Entregar el código al candidato (impreso o en pantalla).

El candidato lee 2 minutos en silencio, luego:

1. Explica qué hace el código.
2. Identifica todos los problemas.
3. Corrige cada uno explicando el razonamiento.

NO puede usar buscadores, IDE ni asistentes de IA.

El entrevistador puede hacer preguntas mientras el candidato explica.

CÓDIGO A ENTREGAR AL CANDIDATO (imprimir o mostrar en pantalla):

```
// Web API Controller – Sistema de Órdenes de Compra, Oxiquim
[HttpGet("orders")]
public async Task<IActionResult> GetOrdersByUser(string userId)
{
    string sql = $"SELECT Id, Total, Status " +
                $"FROM Orders WHERE UserId = '{userId}'";    // (A)

    using var conn = new SqlConnection(_connectionString);
    conn.Open();                                           // (B)

    using var cmd = new SqlCommand(sql, conn);
    var reader = cmd.ExecuteReader();                     // (C) (D)

    var result = new List<OrderDto>();
    while (reader.Read())                                // (E)
```

```

    {
        result.Add(new OrderDto
        {
            Id      = (int)reader["Id"],
            Total   = (decimal)reader["Total"],
            Status  = reader["Status"].ToString()
        });
    }

    return Ok(result);
}

```

BUGS A IDENTIFICAR (solo entrevistador – NO mostrar al candidato):

BUG A – SQL Injection

Línea: WHERE UserId = '{userId}'
 Problema: userId se concatena directamente. Un atacante envía ' OR '1'='1 y obtiene las órdenes de todos los usuarios.
 Corrección: Usar parámetros:
 string sql = "SELECT Id, Total, Status " +
 "FROM Orders WHERE UserId = @userId";
 cmd.Parameters.AddWithValue("@userId", userId);
 Impacto: CRÍTICO – vulnerabilidad de seguridad de primer orden.

BUG B – Bloqueo del thread (conn.Open)

Línea: conn.Open()
 Problema: En un método async, usar la versión síncrona bloquea el thread del ThreadPool, eliminando el beneficio de la asincronía y causando problemas de escalabilidad bajo carga.
 Corrección: await conn.OpenAsync()
 Impacto: Performance – degradación bajo carga concurrente.

BUG C – Bloqueo del thread (ExecuteReader)

Línea: cmd.ExecuteReader()
 Problema: Mismo problema que B: bloquea el thread mientras espera BD.
 Corrección: await cmd.ExecuteReaderAsync()
 Impacto: Performance – agrava el problema del BUG B.

BUG D – SqlDataReader sin using (resource leak)

Línea: var reader = cmd.ExecuteReader() [sin using]
 Problema: Si ocurre una excepción dentro del while, el reader no se cierra y la conexión subyacente puede quedar ocupada. El connection pool se agota bajo carga.
 Corrección: using var reader = await cmd.ExecuteReaderAsync();
 Impacto: Estabilidad – agotamiento del connection pool en producción.

BUG E – Bloqueo del thread (reader.Read)

Línea: while (reader.Read())
 Problema: Tercera instancia de sincronía en método async. Bloquea el thread en cada fila leída de la BD.
 Corrección: while (await reader.ReadAsync())
 Impacto: Performance – especialmente notable en queries con muchas filas retornadas.

PUNTUACIÓN DEL CODE CHALLENGE:

Puntuación	
0 - No apto	No identifica ningún bug o confunde síntomas con causas
1 - Básico	Identifica solo SQL Injection (A)
2 - Competente	Identifica A + D (resource leak) + al menos un async bug
3 - Destacado	Identifica los 5 bugs, explica el impacto de cada uno en la producción y propone la corrección correcta

PREGUNTAS DE SEGUIMIENTO para quien termina rápido:

- "¿Cómo validarías que userId no sea nulo o vacío antes de usarlo?"
- "¿Cómo agregarías paginación (page, pageSize) a esta query de forma segura sin SQL Injection?"
- "Si esta query se ejecuta 500 veces por segundo, ¿qué cambiarías en la arquitectura para reducir la carga sobre SQL Server?"

SECCIÓN H – SCORECARD RESUMEN

TABLA DE PUNTUACIÓN (cada pregunta: 0-3 puntos)

Sección	RE	Pregunta	Peso
Backend	1	A-1: C# async/await + .NET Core vs Framework	12%
Backend	1	A-2: Web API + WCF + seguridad identidad	8%
Backend	1	A-3: VB.NET legacy + migración gradual	5%
Base Datos	3	B-1: SQL Server – optimización query non-sargable	12%

	Base Datos	3	B-2: SP + Triggers + Deadlocks	
10%				
	Base Datos	3	B-3: Oracle / PL/SQL / AUTONOMOUS_TRANSACTION	
5%				
	Arq.	4	C-1: Diseño multicapas en vivo con interrupciones	
10%				
	Arq.	4	C-2: Strategy pattern para generación de reportes	
8%				
	Integr.	5	D-1: webMethods + orquestación + idempotencia	
8%				
	Integr.	5	D-2: REST vs SOAP – criterio técnico real	
5%				
	Frontend	6	E-1: JS/jQuery performance + debounce	
5%				
	Frontend	6	E-2: Bootstrap responsive + CSS Grid	
3%				
	Herr.	2	F-1: Azure DevOps/TFS pipeline CI/CD	
4%				
	Código	1+3	G: Code Challenge C# async + SQL Injection (5 bugs)	
5%				
	+-----+			
	-----+			
	TOTAL			
100%				
	+-----+			
	-----+			

Nota: umbral Semi Senior menor que Senior (70% vs 75%) porque se valora el potencial de crecimiento además del dominio técnico actual.

X Desconoce completamente async/await en C#.
X No conoce el sistema de grilla de Bootstrap.

ADAPTABILIDAD CULTURAL (Oxiquim valora tecnicismo + adaptación cultural):

¿Demostró apertura pragmática a trabajar con sistemas legacy (.NET Framework, VB.NET, WCF, webMethods, TFS)?

- ☐ Sí - actitud pragmática y sin prejuicios ante legacy
☐ Sí - pero con resistencia o comentarios negativos al legacy
☐ No - rechaza trabajar con tecnologías fuera del stack moderno

NOTAS DEL ENTREVISTADOR:

DECISIÓN FINAL: ☐ Recomendado ☐ A evaluar ☐ No apto

Entrevistador: _____ Fecha: _____

Challenge generado: 2026-05-11 - TechChallenge Architect | ACL Chile
Diseño anti-IA: follow-ups verbales sorpresa · diseño en vivo con interrupciones · código con 5 bugs deliberados (SQL Injection + async/await + resource leak) · preguntas de experiencia real en proyectos concretos .
tecnologías especializadas (webMethods, WCF, AUTONOMOUS_TRANSACTION PL/SQL)
que exponen inmediatamente las respuestas genéricas de un asistente IA
=====